

Session 03 - Optional Params, Function Return, Overloading, Enums, Promises

Naveen AutomationLabs | 2026 Playwright Batch G1

1. Optional Parameters

In TypeScript, you can mark a parameter as optional using the ? symbol. Optional parameters must come after required parameters.

```
function printData(name: string, age?: number): void {
  console.log(name);
  console.log(age);
}

printData('Peter'); // Peter, undefined
printData('Peter', 30); // Peter, 30
```

Practical example -- browser launch:

```
function launchBrowser(browser: string, headless?: boolean): boolean {
  if (headless) {
    console.log('run tcs in', browser, 'in headless mode:', headless);
    return true;
  } else {
    console.log('run tcs in normal mode with:', browser);
    return true;
  }
}

let isLaunched = launchBrowser('chrome', true);
console.log(isLaunched);
```

Combining normal, optional, and rest parameters:

```
// Order: normal params -> optional params -> rest params
function search(name: string, color: string,
  price?: number, seller?: string, ...values: any): void {

  if (price && seller) {
    console.log('performing search:', name, color, price, seller);
  } else {
    console.log('performing search:', name, color);
  }
}

search('macbook pro', 'white', 1000, 'iplanet pvt ltd');
search('macbook pro', 'white');
```

2. Function Return Types

In TypeScript, you specify the return type of a function after the parentheses using a colon.

```
function launchBrowser(name: string): boolean {
  switch (name.toLowerCase().trim()) {
```

```

case 'chrome':
console.log('chrome is launched');
return true;
case 'firefox':
console.log('ff is launched');
return true;
default:
console.log('invalid browser');
return false;
}
}

let isLaunched = launchBrowser('ie');
console.log(isLaunched); // false

```

3. Functions with Promises (async/await)

When a function returns a Promise, you specify the return type as `Promise<type>`. This is very common in Playwright.

```

function getNumber(): Promise<number> {
return Promise.resolve(100);
}
getNumber().then((res) => console.log(res)); // 100

function getTrainerName(): Promise<string> {
return Promise.resolve('naveen');
}
getTrainerName().then((tr) => console.log(tr)); // naveen

```

Promise with custom type and setTimeout:

```

type userType = { name: string, age: number };

function getUserData(): Promise<userType> {
return new Promise((resolve, reject) => {
setTimeout(() => {
let user = { name: 'Anuja', age: 30 };
resolve(user);
}, 3000);
});
}

// Using async/await:
async function getMyUserData() {
let myUser = await getUserData();
console.log(myUser);
}
getMyUserData();

```

Playwright-style Promise:

```

function click(element: string): Promise<void> {
console.log('click on', element);
return Promise.resolve(); // no value returned
}

async function doClick(element: string): Promise<void> {
await click(element);
}

```

```
// In test code:  
doClick('forgotPasswordLink');
```

In Playwright, most actions like `click()`, `fill()`, `goto()` return `Promise<void>`. You always use `'await'` with them.

4. Function Overloading

TypeScript supports function overloading. You define multiple signatures (prototypes) but only one implementation with the body.

```
// Step 1: Define signatures (no body):  
function combination(a: number, b: number): number;  
function combination(a: string, b: number): string;  
function combination(a: string, b: string): string;  
function combination(a: number, b: string): string;  
  
// Step 2: One implementation with 'any' types:  
function combination(a: any, b: any): any {  
  return a + b;  
}  
  
// Step 3: Call with matching signatures:  
console.log(combination(100, 200)); // 300  
console.log(combination('tom', 10)); // 'tom10'  
console.log(combination('tom', 'pop')); // 'tompop'  
console.log(combination(100, 'pop')); // '100pop'
```

Unlike JavaScript where the last function wins, TypeScript function overloading gives you compile-time checking for each signature.

5. Enums

Enums let you define a set of named constants. They make your code more readable and prevent typos in hardcoded string values.

```
enum BROWSERS {  
  CHROME = 'chrome',  
  FIREFOX = 'firefox',  
  EDGE = 'edge',  
  SAFARI = 'safari'  
}  
  
enum ENVS {  
  QA = 'qa',  
  STAGE = 'stage',  
  PROD = 'prod'  
}  
  
console.log(BROWSERS.CHROME); // 'chrome'  
console.log(BROWSERS.FIREFOX); // 'firefox'
```

Using enums in a test file:

```
import { BROWSERS, ENVS } from './enumTest';  
  
let browserName = 'chrome'; // from config file  
  
switch (browserName) {
```

```
case BROWSERS.CHROME:  
  console.log('open chrome');  
  break;  
case BROWSERS.FIREFOX:  
  console.log('open ff');  
  break;  
default:  
  break;  
}
```

Export enums using 'export { BROWSERS, ENVS }' and import them in other files. This keeps your constants centralized and reusable.